

I 1 Chọn biểu diễn 3D, độ sâu và SLAM (3D Representations, Depth and SLAM)

PHỤ THUỘC C/16-thi-giac-3d/01-bon-the-he-3d (bốn thế hệ); B/10-thiet-ke-kien-truc (thiết kế theo bất biến); B/07-chia-du-lieu-va-danh-gia (ATE/RPE và giao thức đánh giá); A/01-toan-toi-thieu/03-xac-suat-va-thong-tin (tính xác định được của tham số).

TÓM TẮT

Mục này nói về ba quyết định con nằm bên trong mọi hệ thống 3D: lưu cảnh bằng biểu diễn nào, lấy độ sâu từ đâu, và học sâu làm gì được trên dữ liệu không phải lưới. Đọc xong bạn làm được ba việc. Chọn biểu diễn cho một pipeline mới mà không phải viết lại nó sáu tháng sau. Nhìn một mô-đun độ sâu là biết nó có quyền xuất hiện trong phần dư metric hay không. Và đọc một mạng trên point cloud để biết nó mù với cái gì. Nếu chỉ nhớ hai điều: biểu diễn quyết định *phép toán nào rẻ*, nên phải chọn nó theo khâu cuối cùng của pipeline chứ không theo khâu đầu tiên. Và độ sâu đơn ảnh không phải phép đo mà là suy đoán có học, với một tỉ lệ trôi giữa các khung hình — đó chính là chỗ nó phá hỏng một bộ ước lượng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **voxel** — ô lập phương trong một lưới đều 3D — “pixel của không gian ba chiều”. Đơn giản nhưng số ô tăng theo lũy thừa ba của độ phân giải.
- **point cloud (point cloud)** — một tập tọa độ 3D không có thứ tự và không có kết nối giữa các điểm; đây là dạng dữ liệu thô của LiDAR và của stereo dày đặc.
- **mesh (mesh)** — biểu diễn bề mặt bằng các đỉnh nối thành tam giác; chuẩn của đồ họa máy tính, vì nó cho biết *đâu là mặt* chứ không chỉ *đâu có vật chất*.
- **SDF (signed distance function)** — một hàm gán cho mỗi điểm trong không gian khoảng cách có dấu tới bề mặt gần nhất — âm ở trong, dương ở ngoài. Bề mặt chính là tập điểm có giá trị 0, nên nó được định nghĩa rất sạch.
- **trường ẩn (implicit field)** — cách mô tả hình dạng bằng một *hàm* (thường là một mạng nhỏ) thay vì bằng một danh sách phần tử; SDF là ví dụ điển hình.
- **đường cơ sở (baseline)** — khoảng cách giữa hai tâm camera trong một hệ stereo; nó là thứ duy nhất cấp đơn vị mét cho phép đo độ sâu.
- **ATE và RPE** — hai chỉ số đánh giá quỹ đạo: ATE đo sai lệch tuyệt đối sau khi căn chỉnh toàn cục hai quỹ đạo, RPE đo sai lệch trên các đoạn ngắn nên không cần căn chỉnh. Bạn đã quen hai chỉ số này; ở đây chỉ dùng chúng làm ví dụ về việc “phép căn chỉnh che giấu điều gì”.
- **bất biến hoán vị (permutation invariance)** — tính chất của một hàm mà kết quả không đổi khi ta xáo trộn thứ tự các phần tử đầu vào; bắt buộc phải có nếu đầu vào là một *tập*.

A. Đặt vấn đề

Trong ảnh 2D không ai phải chọn biểu diễn: ảnh là một lưới pixel, hết chuyện. Trong 3D thì không có biểu diễn mặc định nào cả, và lựa chọn đó quyết định nhiều hơn cả việc chọn kiến trúc mạng. Lý do rất cụ thể: **biểu diễn quyết định phép toán nào rẻ**. Trên lưới *voxel*, tích chập là rẻ nhưng bộ nhớ tăng theo lũy thừa ba. Trên *point cloud*, bộ nhớ rẻ nhưng “lân cận” không có sẵn nên mọi phép cục bộ phải trả tiền tìm kiếm. Trên *mesh*, bề mặt là tường minh nên render và mô phỏng vật lý rẻ, nhưng thay đổi *topology* thì gần như không khả vi. Một khi đã chọn biểu diễn, bạn đã chọn luôn tập bài toán còn giải được hiệu quả — và đây là quyết định khó đảo ngược nhất trong một pipeline 3D.

Câu chuyện thứ hai là nguồn của độ sâu. Bốn thế hệ ở mục trước đều cần độ sâu từ đâu đó. Có ba cách lấy nó: một ảnh, hai ảnh, hoặc cảm biến chủ động. Chúng khác nhau không phải ở độ chính xác mà ở chỗ *cái nào xác định được tỉ lệ tuyệt đối*. Cuối cùng, ta đặt SLAM và VIO vào đúng

vị trí: không phải một chủ đề riêng mà là hệ thống duy nhất bắt buộc giải cả ba bài toán — biểu diễn, độ sâu, tối ưu — dưới ràng buộc thời gian thực.

Học mục này thì làm được gì mà trước đó không làm được? Ba việc: chọn được biểu diễn cho một pipeline mới mà không phải viết lại nó sáu tháng sau; nhìn một mô-đun độ sâu và nói ngay nó có quyền xuất hiện trong phần dư metric hay không; và đọc được một mô hình học trên point cloud để biết nó mù với cái gì.

TRỰC GIÁC

Chọn biểu diễn 3D giống chọn hệ toạ độ để viết một bài toán cơ học. Bài toán không đổi, nhưng viết trong hệ Descartes thì trọng lực đơn giản còn chuyển động tròn phức tạp; trong hệ cực thì ngược lại. Voxel là hệ Descartes của 3D: đều đặn, dễ lập trình, và lãng phí khủng khiếp vì phần lớn không gian là rỗng. Point cloud là “danh sách các chỗ có vật chất”: gọn đúng bằng lượng thông tin thật, nhưng bạn phải tự dựng lại khái niệm hàng xóm mỗi lần cần tới nó.

B. Năm biểu diễn và cái giá của mỗi cái

Năm biểu diễn dưới đây không ngang hàng nhau. Chúng chia làm ba nhóm theo *nguồn gốc*, và biết nhóm nào giúp đoán ngay chỗ hỏng:

- **Biểu diễn đo được** — voxel và point cloud, ra thẳng từ cảm biến. Chỗ hỏng của chúng là chỗ hỏng của cảm biến: mật độ không đều, vùng khuất, nhiễu phụ thuộc khoảng cách.
- **Biểu diễn đồ hoạ** — mesh. Chỗ hỏng là bước tái dựng bề mặt đứng trước nó.
- **Biểu diễn được tối ưu** — **trường ẩn** (*implicit field*) và *Gaussian splatting*. Chúng không tồn tại cho tới khi chạy xong một vòng tối ưu trên chính cảnh đó, nên chỗ hỏng của chúng là chỗ hỏng của hàm mất mát đã sinh ra chúng.

Bảng 1. Năm biểu diễn 3D. Cột cuối là cột quyết định: mỗi biểu diễn hỏng ở một chỗ khác nhau, và chỗ đó thường lộ ra mụn.

Biểu diễn	Cơ chế lưu	Được	Mất / hỏng khi
Voxel	Lưới đều 3D, mỗi ô một giá trị	Conv 3D dùng được nguyên xi; lân cận cho sẵn	Bộ nhớ theo lũy thừa ba; hỏng ngay khi cần độ phân giải cao
Point cloud	Tập điểm không thứ tự	Gọn, hợp với LiDAR và stereo	Không có cấu trúc lân cận; phải bất biến hoán vị; mật độ không đều
Mesh	Đỉnh + mặt	Chuẩn của đồ hoạ và mô phỏng; bề mặt tường minh	Tối ưu topology gần như không khả vi; mặt tự cắt nhau
Implicit field (SDF, occupancy)	Một hàm $\mathbb{R}^3 \rightarrow \mathbb{R}$, thường là MLP	Độ phân giải liên tục; bề mặt là tập mức 0, định nghĩa sạch	Render chậm vì phải lấy mẫu nhiều lần dọc tia; khó chỉnh sửa cục bộ
Gaussian splatting	Tập Gaussian dị hướng có màu và độ đục	Rasterize được nên render rất nhanh	Dung lượng lớn; hình học không tường minh nên trích mesh khó

Hình 1 vẽ cùng một mặt cong bằng cả năm biểu diễn, đặt cạnh nhau — đây là cách nhanh nhất để thấy mỗi cái giữ lại gì và vứt đi gì.

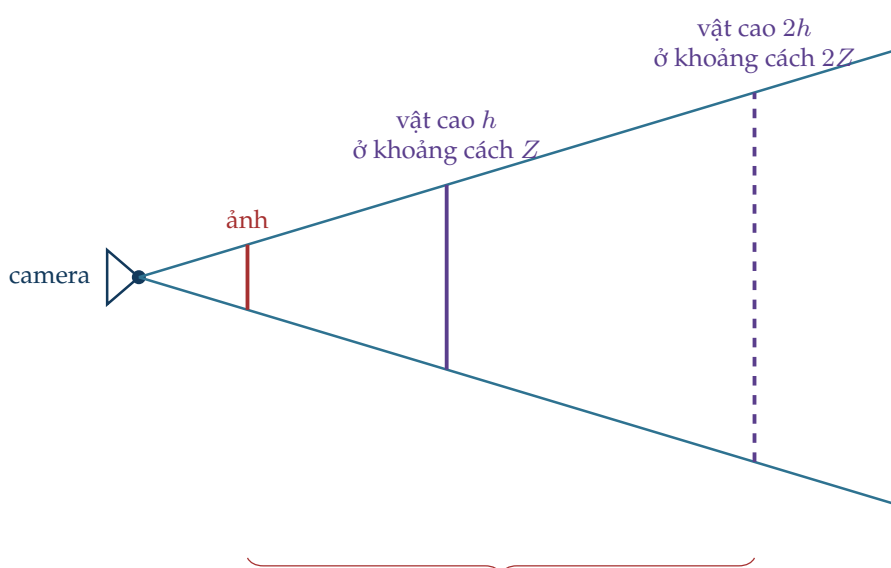


Hình 1. Cùng một mặt cong, năm biểu diễn. Nhìn hàng chú thích dưới cùng thì thấy ngay điều mà bảng không nói được: mỗi biểu diễn giữ lại một loại thông tin và vứt đi loại khác. Point cloud biết ở đâu có vật chất nhưng không biết đâu là mặt; voxel biết cả không gian rỗng nhưng trả tiền cho nó; mesh biết mặt nhưng khoá cứng topology; SDF định nghĩa mặt sạch nhất nhưng phải dò dọc tia mới render được; Gaussian render nhanh nhất nhưng không có mặt để mà trích ra.

Con số làm cho dòng đầu bảng thành cụ thể. Một lưới 512^3 chứa khoảng $1,34 \times 10^8$ ô; lưu một số thực 32-bit cho mỗi ô đã tốn 512 MiB, và một tầng conv 3D với 32 kênh ở cùng độ phân giải cần 16 GiB chỉ để giữ *activation*. Nâng lên 1024^3 thì nhân tám. Trong khi đó một quét LiDAR dày đặc của cùng cảnh có chừng 2×10^5 điểm, tức 2,4 MB nếu lưu ba toạ độ float — ít hơn hai bậc độ lớn. Toàn bộ dòng **tích chập thưa** (*sparse convolution*) tồn tại chỉ vì khoảng cách này: hơn 99 % ô của lưới đều là rỗng, nên trả tiền cho chúng là trả tiền cho hư không.

C. Độ sâu đến từ đâu, và cái gì cố định tỉ lệ

Với ảnh đơn, bài toán độ sâu không phải khó — nó *không xác định được*. Nhân toàn bộ cảnh với hệ số s và nhân luôn mọi vị trí camera với s thì mọi ảnh thu được giống hệt nhau. Hình 2 là toàn bộ chứng minh, vẽ ra trong một hình.



Hai cảnh khác nhau, **một ảnh duy nhất**. Không phép đo nào trong ảnh phân biệt được chúng.

Hình 2. Tính không xác định được của tỉ lệ trong ảnh đơn. Nhân cảnh với s và nhân quãng đường camera với cùng s cho ra đúng bộ ảnh cũ; vì vậy *không lượng dữ liệu nào* chứa được, chỉ prior mới chứa được. Đây cũng chính là lý do SLAM đơn mắt có một bậc tự do tỉ lệ, và vì sao chỉ cần thêm một gia tốc kế — một cảm biến biết đơn vị mét — là bậc tự do đó biến mất.

Đây đúng là hiện tượng **không xác định được** (*non-identifiability*) ở chương 4: dữ liệu không chứa thông tin để phân biệt hai giả thuyết.

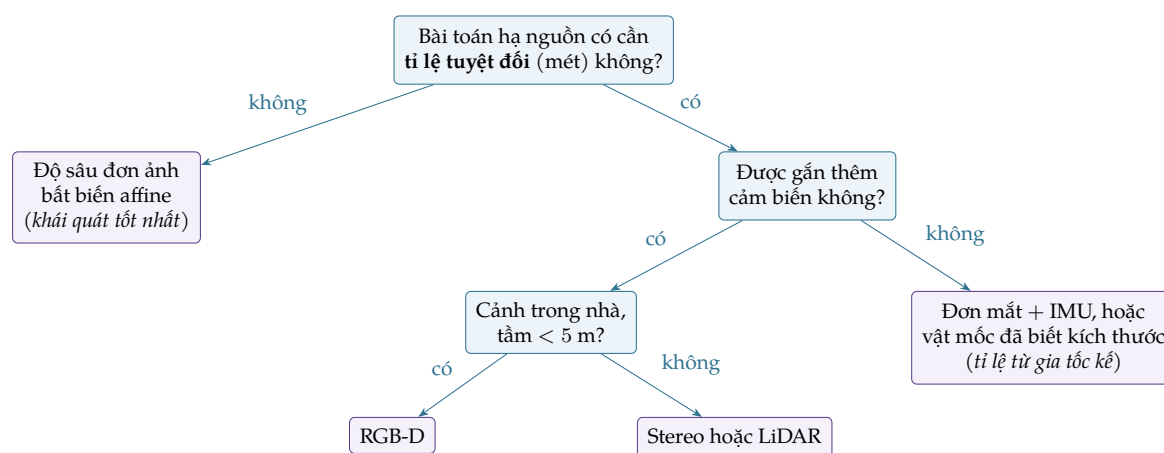
↪ **trường hợp riêng của B/04** — scale ambiguity là một ca sạch của tính không xác định được: hai tham số khác nhau sinh ra cùng một phân phối quan sát, nên thêm dữ liệu không giúp gì.

Một mạng độ sâu đơn ảnh phá thế bế tắc bằng cách học kích thước điển hình của vật thể và mối liên hệ với tiêu cự; nghĩa là con số nó trả về là một *suy đoán có học*, không phải một phép đo. Nó vì thế sai theo kiểu rất đặc trưng ở đúng những cảnh phá vỡ prior kích thước: mô hình thu nhỏ, ảnh chụp từ trên cao, nội soi, ảnh macro.

Bốn nguồn độ sâu, đọc theo cái mà mỗi nguồn dùng để phá thế bế tắc:

- **Đơn ảnh, metric** — phá bằng prior kích thước. Cho ra mét ngay, nhưng chỉ đúng trong miền đã học.
- **Đơn ảnh, bất biến affine** — không phá, mà *bỏ* tỉ lệ đi: huấn luyện trên nhiều tập dữ liệu hỗn tạp với hàm mất mát bất biến với phép biến đổi affine của độ sâu, nên mô hình chỉ phải học thứ tự và hình dạng tương đối [1], [2]. Khái quát rất tốt; đổi lại mỗi khung hình còn dư hai ẩn (tỉ lệ và độ dịch) và hai ẩn đó *không nhất quán giữa các khung hình*.
- **Stereo** — phá bằng vật lý: đường cơ sở đã biết cho tỉ lệ. Cần hiệu chỉnh chính xác, cần texture để khớp, và có chân trời độ sâu tính được từ $\Delta Z = Z^2 \Delta d / (fb)$.
- **RGB-D** — phá bằng cảm biến chủ động: chiếu ánh sáng ra và đo phản hồi, nên có tỉ lệ tuyệt đối ngay cả trên tường trắng. Chết dưới nắng, trên mặt gương, trên vật hấp thụ hồng ngoại; có sai lệch phụ thuộc khoảng cách cần hiệu chuẩn riêng.

Hình 3 gói bốn lựa chọn đó thành một cây quyết định dùng được ngay.



Hình 3. Cây quyết định chọn nguồn độ sâu. Câu hỏi đầu tiên không phải “mô hình nào chính xác nhất” mà “bài toán của tôi có thật sự cần mét không”. Rất nhiều bài toán chỉ cần độ sâu tương đối: tránh vật cản gần, sắp thứ tự lớp, tách nền. Ở đó lựa chọn bất biến affine vừa rẻ vừa bền hơn hẳn.

D. SLAM và VIO đứng ở đâu

Bạn đọc chương này đã sống trong SLAM nên mục này không nhắc lại cơ chế; nó chỉ đặt SLAM vào bức tranh bốn thể hệ. SLAM là thể hệ một chạy dưới ràng buộc thời gian thực, và toàn bộ kiến trúc *frontend/backend* là hệ quả trực tiếp của ràng buộc đó: frontend phải quyết định tham lam trong vài mili-giây mỗi khung hình nên chỉ dùng ràng buộc cục bộ, còn backend là nơi hình học thực sự được ép đúng trên một cửa sổ hoặc trên toàn đồ thị [3], [4]. **Đóng vòng** (*loop closure*) không phải một tính năng phụ mà là thứ duy nhất đối *bậc* của sai số: không có nó, sai số tích lũy tăng đơn điệu theo quãng đường; có nó, sai số bị chặn bởi độ chính xác của phép đóng vòng.

Bốn thể hệ chạ vào SLAM ở ba chỗ khác nhau:

- **Thể hệ lai thay frontend** — đặc trưng và khớp học được, backend giữ nguyên. Đây là hướng đã thành thực dụng.

- **Thế hệ lai, bản triệt để hơn** — dòng DROID-SLAM dùng **dòng quang** (*optical flow*) dày đặc học được nhưng vẫn nhét một tầng hiệu chỉnh chùm tia khả vi vào giữa, nên nó vẫn là thế hệ hai chứ không phải thế hệ bốn.
- **Thế hệ ba thay bản đồ** — giữ tracker cổ điển nhưng lưu bản đồ bằng trường bức xạ hoặc Gaussian. Đánh đổi bản đồ thừa lấy khả năng render: đáng giá cho robot cần nhìn lại cảnh, đáng ngờ cho robot chỉ cần định vị.

Về đánh giá chỉ cần một câu nhắc: *ATE* cần một phép căn chỉnh toàn cục trước khi đo, vì quỹ đạo ước lượng sống trong một hệ toạ độ tự do — với đơn mắt thì tự do cả tỉ lệ. Chính phép căn chỉnh đó hấp thụ một phần *drift* toàn cục. *RPE* thì đo trên các đoạn ngắn nên không cần căn chỉnh, và vì thế nhạy với lỗi cục bộ mà *ATE* làm phẳng đi [5]. Hai chỉ số phải đọc cùng nhau, và mọi thảo luận sâu hơn về giao thức — chia dữ liệu, số lần lặp, khoảng tin cậy — nằm ở B/07-chia-du-lieu-va-danh-gia.

↪ **ràng buộc bởi B/07** — phép căn chỉnh của *ATE* là một bước tiền xử lý của giao thức đánh giá, nên nó quyết định lỗi nào còn nhìn thấy được.

E. Học sâu trên point cloud

Muốn đưa mạng nơ-ron lên point cloud thì vấp ngay vào một trở ngại cấu trúc: đầu vào là một *tập*, không phải chuỗi hay lưới. Với n điểm có $n!$ cách đánh số cùng mô tả một hình dạng, nên bất kỳ hàm nào không bất biến với hoán vị đều đang học lại cùng một hình dạng hàng giai thừa lần.

I. Mạch 2 — từ tập điểm tới mạng dùng được

- Vấn đề** đầu vào là tập không thứ tự, mà MLP thường thì không bất biến hoán vị.

Giải pháp ép mọi điểm qua cùng một MLP chia sẻ trọng số rồi gộp bằng một **hàm đối xứng** (*symmetric function*) — lấy max theo từng chiều đặc trưng [6].

Giả định hình dạng mô tả được đủ tốt bằng một vectơ đặc trưng toàn cục.

Hệ quả bất biến hoán vị ở đây là bất biến *theo cấu trúc*, không phải học được từ **tăng cường dữ liệu** (*data augmentation*), nên nó đúng tuyệt đối chứ không đúng gần đúng.
- Vấn đề** max toàn cục vứt bỏ mọi cấu trúc cục bộ: đầu ra chỉ phụ thuộc một tập điểm “tới hạn” rất nhỏ — những điểm đang giữ giá trị lớn nhất ở mỗi chiều.

Giá phải trả thêm hay bớt điểm bên trong bao lồi của tập tới hạn không đổi đầu ra: rất bền với thay đổi mật độ, nhưng mù với chi tiết bề mặt mịn.

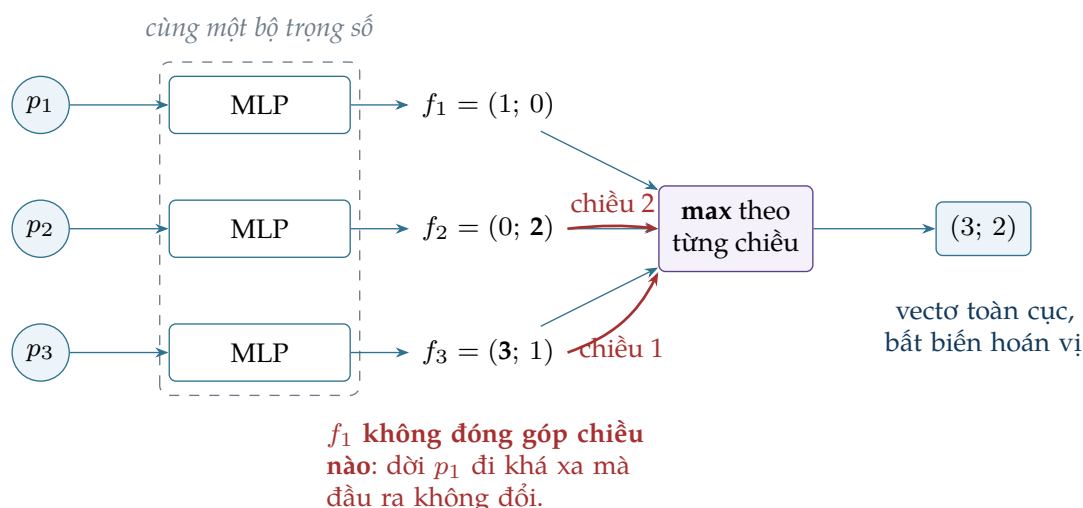
Giải pháp lồng PointNet vào chính nó theo phân cấp không gian — nhóm điểm theo bán kính, chạy PointNet trên từng nhóm, rồi lặp lại ở tỉ lệ lớn hơn [7].

Vấn đề mới chi phí truy vấn lân cận thành nút cổ chai, và kết quả nhạy với việc chọn bán kính.
- Vấn đề** ở quy mô một cảnh đường phố với hàng trăm nghìn điểm, truy vấn lân cận là quá đắt.

Giải pháp quay lại lưới nhưng chỉ giữ ô không rỗng (*sparse convolution*), hoặc gộp điểm theo cột dọc thành *pillar* rồi rơi về một bài toán 2D quen thuộc [8].

Giả định cảnh ngoài trời gần như phẳng theo phương thẳng đứng nên nén trục z mất ít thông tin.

Giá phải trả giả định đó sai ngay trong nhà, nơi vật thể chồng lên nhau theo chiều cao.



Hình 4. Hàm đối xứng của PointNet, vẽ trên đúng ba điểm của ví dụ tính tay. Mọi điểm đi qua cùng một MLP rồi được gộp bằng phép max theo từng chiều, nên xáo trộn thứ tự ba điểm cho ra cùng một vector $(3; 2)$ — tính bất biến ở đây là bất biến theo cấu trúc, không phải học được. Nhưng hình cũng phơi ra cái giá: đầu ra chỉ do *tập điểm tới hạn* quyết định (ở đây là f_2 và f_3), còn f_1 hoàn toàn vô hình. Đúng cái mẹo giữ cho mạng bất biến cũng là đúng cái mẹo làm nó mù.

Hình 4 vẽ đúng cơ chế đó trên ba điểm.

Tính tay để thấy hàm đối xứng thật sự làm gì. Lấy ba điểm đã qua MLP thành các đặc trưng hai chiều $f_1 = (1; 0)$, $f_2 = (0; 2)$, $f_3 = (3; 1)$. Gộp bằng max theo từng chiều cho $(3; 2)$. Đảo thứ tự ba điểm bằng bất kỳ hoán vị nào vẫn cho $(3; 2)$ — đó là toàn bộ mẹo của PointNet. Nhưng phép tính đó cũng phơi ra ngay giới hạn: chiều thứ nhất chỉ do f_3 đóng góp, chiều thứ hai chỉ do f_2 ; điểm f_1 hoàn toàn không ảnh hưởng, và bạn có thể dời nó đi khá xa mà vector toàn cục không nhúc nhích. *Đúng cái mẹo giữ cho mạng bất biến cũng là đúng cái mẹo làm nó mù.* Cùng cơ chế này sẽ xuất hiện lại ở C/17-mo-hình-sinh/02-toi-uu-tren-pixel, nơi ma trận Gram vứt bỏ vị trí không gian để giữ lại “phong cách” — hai kỹ thuật trông không liên quan, một nguyên lý.

→ cùng nguyên lý với C/17/02 — ma trận Gram gộp trên vị trí đúng như PointNet gộp trên điểm: một hàm đối xứng mua tính bất biến bằng chính thông tin bị gộp đi.

→ trường hợp riêng của B/10 — bất biến hoán vị là một ví dụ của việc cài thiên lệch quy nạp thẳng vào kiến trúc thay vì học nó từ dữ liệu.

F. Giới hạn và trường hợp hỏng

Điểm yếu chung của cả mục này là các biểu diễn không chuyển đổi qua lại mà không mất mát. Điểm sang mesh cần một bước tái dựng bề mặt vốn tự nó là bài toán ước lượng có prior riêng; mesh sang SDF cần mesh kín, điều gần như không đúng với mesh quét thực tế; Gaussian sang mesh là bài toán mở. Vì vậy lỗi thiết kế đắt nhất trong một pipeline 3D là chọn biểu diễn theo khâu đầu tiên thay vì theo khâu cuối cùng cần dùng kết quả.

Hai chế độ hỏng đáng nhớ riêng:

- **Độ sâu học được:** nguy hiểm nhất không phải sai số lớn mà là sai số *không nhất quán theo thời gian* — mỗi khung hình một tỉ lệ khác nhau trong khi từng khung hình đều trông hợp lý.
- **Point cloud:** thay đổi mật độ giữa lúc huấn luyện và lúc chạy. Mô hình huấn luyện trên LiDAR 64 tia thường tụt mạnh trên 32 tia, và triệu chứng là *mất vật ở xa trước tiên* — đúng nơi mật độ giảm nhanh nhất.

CẠM BẦY

Cạm bẫy quen thuộc với người làm SLAM: lấy độ sâu đơn ảnh làm phép đo trong một bộ ước lượng có trọng số. Nó không những sai tỉ lệ, mà tỉ lệ đó còn *trôi giữa các khung hình*. Sai số của nó vì thế có tương quan mạnh theo thời gian. Đó là vi phạm đúng giả định nhiều trắng độc lập mà bộ lọc dựa vào, và hệ quả là hiệp phương sai bị đánh giá quá lạc quan. Có hai cách dùng an toàn. Cách thứ nhất: coi nó như ràng buộc *tương đối* bên trong một khung hình, ví dụ thứ tự độ sâu hay tính phẳng của mặt. Cách thứ hai: ước lượng tường minh một tỉ lệ và một độ dịch cho mỗi khung hình, rồi đưa hai ẩn đó vào bài toán tối ưu. Cách sai là nhét mét thẳng vào phần dư.

CẦN NHỚ

Biểu diễn quyết định phép toán nào rẻ, và vì thế quyết định bài toán nào bạn còn giải được về sau: hãy chọn nó theo khâu *cuối cùng* của pipeline, không theo khâu đầu tiên. Còn về độ sâu: chỉ có baseline và cảm biến chủ động cho ra mét; mọi thứ khác cho ra một suy đoán có học, và suy đoán đó không được phép xuất hiện trong phần dư metric mà không kèm hai ẩn tỉ lệ và độ dịch. [T4]

G. Kết nối

Mục này cung cấp vốn từ cho C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat, nơi hai biểu diễn cuối bảng được đào sâu như hai câu trả lời cho cùng một bài toán render. Bất biến hoán vị của PointNet là trường hợp riêng của nguyên tắc thiết kế theo bất biến ở B/10-thiet-ke-kien-truc; tính không xác định được của tỉ lệ là trường hợp riêng của B/04-nen-tang-hoc-tu-du-lieu. Phần ATE/RPE nổi thẳng sang B/07-chia-du-lieu-va-danh-gia, còn tụt hiệu năng khi đổi mật độ cảm biến là một ca của B/08-dich-chuyen-phan-phoi.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao bộ nhớ voxel không phải vấn đề kỹ thuật có thể tối ưu đi mà là hệ quả cấu trúc của biểu diễn?
2. Độ sâu đơn ảnh bất biến affine tổng quát tốt hơn độ sâu metric. Cái giá của tính tổng quát đó hiện ra ở đâu khi ghép nhiều khung hình?
3. PointNet bất biến hoán vị “theo cấu trúc”. Vì sao điều đó mạnh hơn hẳn việc học tính bất biến bằng augmentation, và nó khiến mạng mù với cái gì?
4. Nếu loop closure bị tắt, sai số của SLAM đổi bậc như thế nào, và chỉ số nào trong hai chỉ số ATE/RPE phản ánh sự thay đổi đó rõ hơn?
5. Một mô hình phát hiện vật thể trên LiDAR 64 tia được đưa sang cảm biến 32 tia. Bạn dự đoán triệu chứng đầu tiên là gì, và vì sao?
6. Hình 2 nói rằng không lượng dữ liệu nào phá được tính không xác định của tỉ lệ. Vậy vì sao một mạng “metric depth” vẫn cho ra mét, và điều đó ngầm giả định gì?

Nguồn

- [1] D. Eigen, C. Puhrsch, and R. Fergus, “Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network,” in *NeurIPS*, 2014.
- [2] R. Ranftl, K. Lasinger, D. Hafner, et al., “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer,” *IEEE TPAMI*, vol. 44, no. 3, 2022.
- [3] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [4] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [5] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [6] C. R. Qi, H. Su, K. Mo, et al., “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation,” in *CVPR*, 2017.
- [7] C. R. Qi, L. Yi, H. Su, et al., “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” in *NeurIPS*, 2017.
- [8] A. H. Lang, S. Vora, H. Caesar, et al., “PointPillars: Fast Encoders for Object Detection from Point Clouds,” in *CVPR*, 2019.